

UACM

Universidad Autónoma
de la Ciudad de México

Nada humano me es ajeno

Academia de Informática Cuauhtepc
Matutino Ingeniería de Software
Construcción y evolución del software
Laboratorio de Ingeniería de Software
Profesor: Emmanuel Vite Sevilla

Nombre de la práctica: Programa uno

Número: 5

Alumno(s)/Equipo:

Axel Javier Rios Sánchez

Evelin Maldonado Espinosa

Jesús Josué Suarez López

Mauricio Charbel Chavez Galindo

Fecha:21/05/2026



Introducción: El objetivo de este proyecto es desarrollar una versión digital y funcional del clásico juego de cartas UNO, enfocándonos en el ciclo de vida del desarrollo y en cómo un sistema debe adaptarse y evolucionar ante los cambios de requerimientos y tecnologías.

Inicialmente, el núcleo de la lógica del juego (como el manejo del mazo, las reglas de validación y los turnos de la CPU) fue desarrollado y testeado en un entorno de escritorio utilizando Apache NetBeans para asegurar que el backend fuera sólido mediante pruebas estructuradas.

¿Qué ha ido bien durante el sprint?

Jesús: Pues la migración a la web marchó con buen ritmo y la mesa de juego se ve correctamente en el navegador. También aprovechamos las pruebas que habíamos hecho antes en NetBeans para armar los algoritmos de validación. Y usar Visual Studio Code con Live Server nos hizo mucho más fácil y rápido trabajar en la interfaz.

Axel: Durante el sprint lo que ha ido bien es que cumplimos con las entregas a tiempo, logramos ponernos de acuerdo en la mayoría de las decisiones y todos los miembros del equipo aportaron sus opiniones de forma activa.

Charbel: Migración exitosa a la web: El cambio de la lógica que ya teníamos en Java (usando Apache NetBeans) hacia la interfaz gráfica en HTML, CSS y JavaScript ha marchado con buen ritmo. La estructura básica de la mesa de juego y las cartas se visualiza correctamente en el navegador.

Aprovechamiento del código previo: No empezamos de cero. Las pruebas de lógica que corrimos en NetBeans nos sirvieron como un excelente borrador para estructurar los algoritmos de validación (por

ejemplo, checar si una carta coincide en color o número con la del centro).

Entorno de desarrollo sólido: El uso de Visual Studio Code nos ha facilitado la integración del frontend gracias a sus extensiones (como *Live Server*), lo que hace que el flujo de diseño y lógica de la interfaz sea mucho más rápido que compilar en un IDE tradicional.

Evelin: Durante el desarrollo del Proyecto del Juego 1, se logró avanzar de forma constante en la creación de las mecánicas principales del juego. A lo largo del semestre se implementaron elementos importantes como el movimiento del personaje, las colisiones, los enemigos, los puntajes y las pantallas principales. También fue positivo que el equipo lograra integrar cada módulo y realizar pruebas para verificar que el juego funcionara correctamente. La colaboración entre los integrantes permitió resolver errores y cumplir con la mayoría de los objetivos establecidos.

¿Qué se puede mejorar?

Jesús: El código en JavaScript se está volviendo un archivo enorme; deberíamos separar la lógica del juego de la parte visual. Y también tenemos que controlar mejor las versiones con Git para no pisarnos los cambios entre nosotros.

Axel: Lo que se puede mejorar es alcanzar una comunicación fluida y constante al 100%, para que todo fluya de manera funcional y las entregas se ajusten exactamente a lo que se solicita.

Charbel: Modularidad en JavaScript: Como nos enfocamos en que la interfaz web funcionara rápido, la lógica en JS se está empezando a volver un archivo enorme y caótico. Necesitamos aplicar principios de la carrera (como clean code o arquitectura por capas) y separar

la lógica del juego (mazo, turnos) de la manipulación del DOM (la interfaz gráfica).

Automatización de pruebas: En NetBeans solíamos hacer pruebas unitarias de la lógica más fácilmente. Ahora que estamos en la fase web, dependemos mucho de probar "a mano" dando clics en el navegador. Deberíamos implementar un framework de pruebas simple para JS para no perder el control de la evolución del software.

Control de versiones más estricto: Al pasar código de NetBeans a VS Code y luego a la web, a veces nos pisamos los cambios entre nosotros. Hay que coordinar mejor las ramas si estamos usando Git para evitar conflictos raros.

Evelin: Se puede mejorar la organización del trabajo y la administración del tiempo. En varias ocasiones algunas funcionalidades se desarrollaron cerca de la fecha de entrega, lo que redujo el tiempo disponible para probar y optimizar el juego. También sería útil llevar una documentación más detallada de los cambios realizados y establecer revisiones periódicas para identificar errores con anticipación. Esto permitiría tener un desarrollo más ordenado y un producto final más pulido

¿Qué ha fallado?

Jesús: Nos confiamos pensando que pasar la lógica de Java a JavaScript iba a ser directo, pero chocamos con la asincronía y el manejo del estado del juego, lo que nos retrasó con el comportamiento de la CPU. tuvimos un bug crítico donde las cartas de la CPU se mezclaban, y el juego se congelaba al robar cartas del mazo.

Axel: Lo que ha fallado es que, en algunos momentos, la falta de comunicación hizo que no entiéramos completamente los

requisitos, lo que provocó que no entregáramos exactamente lo que se pedía y afectó las calificaciones obtenidas.

Charbel: Desfase en la traducción de lógica (Java a JavaScript): Nos confiamos pensando que pasar la lógica de NetBeans a la web sería directo, pero chocamos con la asincronía de JavaScript y el manejo del estado del juego en el navegador. Esto nos retrasó al programar el comportamiento de los oponentes de la CPU.

Sincronización del ciclo de vida del software: Perdimos un poco de tiempo intentando forzar herramientas de NetBeans para el frontend antes de aceptar que VS Code era la mejor opción para esta etapa de HTML/JS. Esa falta de definición en el *stack* de herramientas al inicio del sprint nos costó un par de días de trabajo muerto.

El manejo del "Estado" del juego: Tuvimos un bug crítico donde las cartas de la CPU se mezclaban con las del jugador humano porque la lógica que maneja los turnos en la interfaz se rompía al renderizar el HTML. El juego se congelaba al robar cartas del mazo.

Evelin: Lo que más falló fue la estimación del tiempo necesario para implementar ciertas funciones del juego, ya que algunos errores técnicos tomaron más tiempo del esperado. Además, en algunos momentos hubo dificultades para integrar el trabajo de todos los integrantes, lo que ocasionó conflictos y ajustes de último momento. Finalmente, el tiempo limitado para realizar pruebas completas provocó que algunos detalles y errores menores permanecieran en la versión final del juego.

Charbel, Evelin, Jesús: En conclusión, el desarrollo de este simulador de UNO ha sido un ejercicio fundamental para entender que el software no es algo estático, sino un ente que debe evolucionar. La transición de una lógica probada en Apache NetBeans hacia una interfaz funcional en HTML gestionada desde

Visual Studio Code, nos permitió aplicar principios de Construcción de Software que van más allá de solo escribir código: aprendimos a gestionar la migración de tecnologías, a adaptar algoritmos de un lenguaje a otro y a priorizar la experiencia del usuario final.

Aprendimos que, si la lógica de negocio está bien estructurada, el software puede evolucionar y adaptarse a nuevas interfaces sin perder su integridad. A pesar de los retos técnicos que implicó migrar de un entorno de escritorio a uno web, logramos consolidar un producto que no solo respeta las reglas del juego original, sino que también ofrece una experiencia de usuario fluida y escalable. Este proyecto refuerza nuestra capacidad para seleccionar las herramientas adecuadas en cada etapa del ciclo de vida y nos prepara para enfrentar entornos de desarrollo reales donde la adaptabilidad es la clave del éxito.

Axel: En conclusión, este fue un proyecto muy interesante a nivel personal. Lo desarrollamos completamente desde cero, iterando y evolucionando paso a paso para hacerlo más claro y comprensible. Finalmente, logramos programarlo con una interfaz gráfica que permite visualizar de forma intuitiva todo el funcionamiento del sistema.